```
// ================================================
// EduGraph - Final Complete C++ Program
// Author: Condesa Andrie
// Description: Calculates current standing,
// required average, and generates a grade projection curve.
// ================================================

#include <bits/stdc++.h>
using namespace std;

// Helper function to safely read a double in range
double getDoubleInRange(const string &prompt, double lo = 0.0,
double hi = 100.0) {
    double val;
    while (true) {
        cout << prompt;
        if (!(cin >> val)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            cout << "  Invalid input. Please enter a number.
\n";
            continue;
        }
        if (val < lo || val > hi) {
            cout << "  Please enter a number between " << lo <<
" and " << hi << ".\n";
            continue;
        }
        return val;
    }
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(NULL);

    cout << "================================\n";
    cout << "         EduGraph v1.0         \n";
    cout << "================================\n\n";

    cout << "Enter completed (already graded) assessments:\n";

    int n_completed;
    while (true) {
```

```
        cout << "How many completed assessments? ";
        if (!(cin >> n_completed) || n_completed < 0) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            cout << "  Please enter a valid non-negative
integer.\n";
        } else break;
    }

    vector<double> completed_grades;
    vector<double> completed_weights;

    for (int i = 0; i < n_completed; ++i) {
        cout << "\nCompleted assessment #" << (i + 1) << ":\n";
        double g = getDoubleInRange("  Grade (0–100): ");
        double w = getDoubleInRange("  Weight (as percent, e.g.
20 for 20%): ");
        completed_grades.push_back(g);
        completed_weights.push_back(w);
    }

    cout << "\nEnter upcoming (future) assessments:\n";
    int n_upcoming;
    while (true) {
        cout << "How many upcoming assessments? ";
        if (!(cin >> n_upcoming) || n_upcoming < 0) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            cout << "  Please enter a valid non-negative
integer.\n";
        } else break;
    }

    vector<double> upcoming_weights;
    for (int i = 0; i < n_upcoming; ++i) {
        cout << "  Upcoming assessment #" << (i + 1) << "
weight (as percent): ";
        double w = getDoubleInRange("", 0.0, 100.0);
        upcoming_weights.push_back(w);
    }

    double target_final = getDoubleInRange("\nEnter your target
final grade (0–100): ", 0.0, 100.0);
```

```
    // --- Calculations ---
    double completed_weight =
accumulate(completed_weights.begin(), completed_weights.end(),
0.0);
    double remaining_weight =
accumulate(upcoming_weights.begin(), upcoming_weights.end(),
0.0);
    double total_weight = completed_weight + remaining_weight;

    cout << fixed << setprecision(3);
    cout << "\n--- Summary of Weights ---\n";
    cout << "Completed weight sum: " << completed_weight << "%
\n";
    cout << "Remaining weight sum: " << remaining_weight << "%
\n";
    cout << "Total weight sum:     " << total_weight << "%\n";

    if (total_weight <= 0.0) {
        cout << "\nError: total weight is zero. Cannot compute
grades.\n";
        return 1;
    }

    // Compute weighted sum of completed grades
    double weighted_sum_current = 0.0;
    for (size_t i = 0; i < completed_grades.size(); ++i) {
        weighted_sum_current += completed_grades[i] *
completed_weights[i] / 100.0;
    }

    // Current weighted average (if any completed)
    double current_average_completed = (completed_weight > 0.0)
        ? (weighted_sum_current / completed_weight) * 100.0
        : 0.0;

    cout << "\n--- Current Standing ---\n";
    cout << "Current contribution: " << weighted_sum_current <<
" / 100\n";
    if (completed_weight > 0.0)
        cout << "Weighted average (completed): " <<
current_average_completed << " / 100\n";
    else
        cout << "No completed assessments yet.\n";
```

```
    cout << "\n--- Target Analysis ---\n";
    if (remaining_weight <= 0.0) {
        double final_if_no_more = weighted_sum_current;
        cout << "No remaining assessments.\n";
        cout << "Final grade would be: " << final_if_no_more <<
" / 100\n";
        if (final_if_no_more >= target_final)
            cout << "You already reached your target of " <<
target_final << ".\n";
        else
            cout << "Target not met; no more weights left.\n";
    } else {
        double required_avg = (target_final -
weighted_sum_current) * 100.0 / remaining_weight;

        cout << "To reach your target of " << target_final <<
", you need an average of:\n";
        if (required_avg <= 0.0)
            cout << "  0 or less  target already achieved!\n";
        else if (required_avg > 100.0)
            cout << "  " << required_avg << " (unachievable
exceeds 100)\n";
        else
            cout << "  " << required_avg << " out of 100 on
upcoming assessments.\n";

        // What-if projection table
        cout << "\nWhat-If Projection Table:\n";
        cout << left << setw(18) << "Upcoming Avg" << setw(20)
<< "Projected Final\n";
        cout << string(35, '-') << "\n";
        for (int possible = 0; possible <= 100; possible += 5)
{
            double projected_final = weighted_sum_current +
(possible * remaining_weight / 100.0);
            if (projected_final > 100.0) projected_final =
100.0;
            cout << setw(12) << possible << setw(10) << "" <<
projected_final << "\n";
        }

        // Save to CSV file
        ofstream csv("curve.csv");
        if (csv) {
            csv << "possible_score,projected_final\n";
```

```
            for (int possible = 0; possible <= 100; possible++)
{
                double projected_final = weighted_sum_current +
(possible * remaining_weight / 100.0);
                if (projected_final > 100.0) projected_final =
100.0;
                csv << possible << "," << projected_final <<
"\n";
            }
            csv.close();
            cout << "\nCSV file 'curve.csv' generated
successfully!\n";
            cout << "You can open it in Excel or Google Sheets
to graph your grade projection.\n";
        } else {
            cout << "Error: could not create 'curve.csv'.\n";
        }
    }

    cout << "\n--- Notes ---\n";
    cout << " All weights are treated as percentages (e.g., 20
= 20%).\n";
    cout << " Final grade = S(completed_grade × weight / 100) +
(upcoming_avg × remaining_weight / 100).\n";
    cout << " Total weight does not have to equal 100; results
are proportional.\n";

    cout << "\nThank you for using EduGraph!\n";
    cout << "Stay consistent and reach your goals.\n";

    return 0;
}
```